New chat
My stuff
Gems
Chats
Image To Video Generation
બિટમેનિપ્યુલેશન અને XOR ઓપરેટર
Sort Array by Binary Bit Count
Kanchie Indian Marble Game
Sort Array By Bit Count
Parlay JavaScript Maximum Function ...

Bit Manipulation: AND, OR, XOR operator

Bit manipulation AND OR and XOR operations "what will be the output if 13 ^ 7 is done" in Gujarati language.

જરૂર 12 : 7 નો બિટવાઈઝ OR ઓપરેશન કરવામાં આવે, તો તેનું પરિણામ 15 આવશે.

આની રીતે સમજૂતી છે કે તમામ બિટ:

૧. દશાંકમાંથી બાઈનરીમાં (Binary) માં ફેરવો:

- 12 દશાંકમાં: 1100
- 7 દશાંકમાં: 0111

૨. OR ઓપરેશન અનુસુચિ: OR ઓપરેશનમાં નિયમ છે કે જો બંનેમાં એક પણ બિટ 1 હોય, તો પરિણામ 1 આવે છે.

```
  1100  (12)
| 0111  (7)
  ----
  1111
```

૩. દશાંકમાં રિસલ્ટ (Decimal) માં ફેરવો:

- બાઈનરી 1111 નો દશાંક કિંમત: $1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 4 + 2 + 1 = 15$.

તો, 12 | 7 = 15.

Bit manipulation AND OR and XOR operations "what will be the output if 13 ^ 7"

Show code

જરૂર 13 ^ 7 નો બિટવાઈઝ XOR ઓપરેશન કરવામાં આવે, તો તેનું પરિણામ 11 આવશે.

આની રીતે સમજૂતી છે તમામ બિટ:

૧. દશાંકમાંથી બાઈનરીમાં (Binary) માં ફેરવો:

- 13 દશાંકમાં: 1101
- 7 દશાંકમાં: 0111

૨. XOR ઓપરેશન અનુસુચિ: XOR ઓપરેશનમાં નિયમ છે કે જો બંનેમાં અલગ અલગ બિટ હોવી 1 તો પરિણામ 1 આવે છે. જો બંનેમાં 1 આવે છે જો બંનેમાં સરખા હોય, તો પરિણામ 0 આવે છે.

```
  1101  (13)
^ 0111  (7)
  ----
  1010
```

૩. દશાંકમાં રિસલ્ટ (Decimal) માં ફેરવો:

- બાઈનરી 1011 નો દશાંક કિંમત: $1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11$.

તો, 13 ^ 7 = 11.

What will be the output of 13^13

Show code

જરૂર 13 ^ 13 નો બિટવાઈઝ XOR ઓપરેશન કરવામાં આવે, તો તેનું પરિણામ 0 આવશે.

નીચેનાના નંબર ગમે તેમાં સરખાં કુદરતી નંબર જો આવતી હોય તમામ સરખાં કરી નાંખવાં આવે જે XOR ઓપરેટર આપે, તેનું પરિણામ હંમેશ 0 જ આવે છે (a ^ a = 0).

આની રીતે સમજૂતી છે તમામ બિટ:

૧. દશાંકમાંથી બાઈનરીમાં (Binary) માં ફેરવો:

- 13 દશાંકમાં: 1101

૨. XOR ઓપરેશન અનુસુચિ: XOR ઓપરેશનમાં નિયમ છે કે જો બંનેમાં સરખા હોય તો 0 આવે છે. (અલગ હોય તો 1 આવે છે).

```
  1101  (13)
^ 1101  (13)
  ----
  0000
```

૩. પરિણામ:

- બાઈનરી 0000 નો દશાંક કિંમત: 0.

તો, 13 ^ 13 = 0.

Left shift and Right shift operation

Left Shift and Right Shift operations, what will be the output of the following :-
જરૂર 8

Show code

જરૂર 20 >> 2 નોટેશનલમાં જોઇએ તો બાઈનરીમાં કામવાળા આપણે, તો તેનું પરિણામ 5 આવશે.

આની રીતે સમજૂતી છે તમામ બિટ:

૧. દશાંકમાંથી બાઈનરીમાં (Binary) માં ફેરવો:

- 20 દશાંકમાં: 10100

૨. શિફ્ટ ઓપરેશન અનુસુચિ: જમણી બાજુ શિફ્ટ કરીએ, તો તેનાં જમણા પાસ વાળો (LSB) નીકળી જાય છે અને ડાબે 0 ઉમેરાય છે.

૩. પરિણામને દશાંકમાં (Decimal) માં ફેરવો:

- બાઈનરી 101 નો દશાંક કિંમત: $1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 5$.

નોટેશનલી રીતે કુદરતી ભાષા નિયમ જોઇએ તો હંમેશાં 2^2 વડે ભાગવા સમાન છે જેમાં ૨ એ બેઝિક સંખ્યાની શક્તિ છે.

- $20 \div 2 = \frac{20}{2^2} = \frac{20}{4} = 2.5$
- ફ્લોરિંગ કરતાં બિટ જેવાં 5 આવશે.

તો, 20 >> 2 = 5.

What will be the output of the following :-
32>>3

Show code

જરૂર 32 >> 3 નોટેશનલમાં શિફ્ટ કરીએ બાઈનરીમાં કામવાળા આપણે, તો તેનું પરિણામ 1 આવશે.

આની રીતે સમજૂતી છે તમામ બિટ:

૧. દશાંકમાંથી બાઈનરીમાં (Binary) માં ફેરવો:

- 32 દશાંકમાં: 100000

૨. Left Shift ઓપરેશન અનુસુચિ: ડાબે LSB (LSB) નોટેશનલી જમણાં તરફ ઉમેરાય છે અને તમામ બિટ જમણી બાજુ ખસેડવામાં આવે છે. નોટેશન, આની રીત 3 તો તેની સાથે જમણી બાજુ 0 ઉમેરવું.

```
  100000  (32)
<< 3
  ------
  000000
```

૩. પરિણામને દશાંકમાં (Decimal) માં ફેરવો:

- બાઈનરી 000000 નો દશાંક કિંમત: 0.

તો, 32 << 3 = 0.

How will the representation look if hooked from binary angle
1010101010<<4

Show code

જરૂર 1010101010 ને ડાબેથી કોમા સાથે સરેરાશ જોઇએ તો કેટલા << 4 (જમણા ઉમેરી) ઓપરેશન કરીએ, તો તેનું પરિણામ નીચેના પ્રમાણે આવશે.

આખાયે દુનિયાનાં સમજૂતી:

૧. મૂળ બાઈનરી સંખ્યા: 1010101010 (મૂલ્ય = 682)

૨. શિફ્ટ કરતાં પહેલાં (Left Shift ઓપરેશન): ડાબેથી જમણાં તરફ તમામ બીટ્સને જમણી બાજુ ખસેડવામાં આવે છે અને ડાબેથી ખાલી જગ્યાઓ ભરવામાં આવે છે. આમાં 4 નોટેશનલી ઉમેરવાનું છે, તેથી ચાર 0 ઉમેરવામાં આવે છે.

```
Python
મૂળ સંખ્યા:          1010101010
4 સ્થાન ડાબે ખસેડ્યું: 10101010100
અંતિમ પરિણામ:        10101010100 (દશાંક: 10880)
```

૩. પરિણામ (અંતિમ): 10880

નોટેશન રીતે કુદરતી ભાષા નિયમ જોઇએ તો 1010101010 ને 16 (કારણ કે $2^4 = 16$) વડે ગુણવાં જરૂર છે જેમાં 4 એ બેઝિક સંખ્યાના ઘાતાંકની, અથવા તેનાં પરિણામ ઉમેરાય છે. તેઓના ગુણાંકમાં આવે.

- $682 \times 16 = 10880$

ટૂંકમાં, આ તો આખાયે બીટની, અથવા તેનાં પરિણામ 1010101010 ને 16 (કારણ કે $2^4 = 16$) વડે ગુણવાં જરૂર છે જેમાં 4 એ બેઝિક સંખ્યાના ઘાતાંકની, અથવા તેનાં પરિણામ ઉમેરાય છે. તેઓના ગુણાંકમાં આવે.

Binary Sum

How will the representation look if hooked from binary angle
1010101010<<4

Show code

જો આપણે 1010101010 ને આખાયે કોમા સાથે બાઈનરી માં ફેરવવા જઇએ << 4 (જમણા ઉમેરી) ઓપરેશન કરીએ, તો તેનું પરિણામ નીચેના પ્રમાણે આવશે.

આખાયે દુનિયાનાં સમજૂતી:

૧. મૂળ આખાયે કોમા સાથે: 1010101010 (મૂલ્ય = 682)

૨. શિફ્ટ કરતાં પહેલાં (Left Shift ઓપરેશન): તમામ બીટ્સને જમણી બાજુ ખસેડવામાં આવે છે અને ડાબેથી ખાલી જગ્યાઓ ભરવામાં આવે છે. આમાં 4 નોટેશનલી ઉમેરવાનું છે, તેથી ચાર 0 ઉમેરવામાં આવે છે.

```
Python
મૂળ સ્થાન:           10101010100
4 સ્થાન ડાબે ખસેડ્યું: 10101010100
અંતિમ પરિણામ:        10101010100 (દશાંક: 10880)
```

૩. પરિણામ (અંતિમ): 10880

નોટેશન રીતે કુદરતી ભાષા નિયમ જોઇએ તો 1010101010 ને 16 (કારણ કે $2^4 = 16$) વડે ગુણવાં જરૂર છે જેમાં 4 એ બેઝિક સંખ્યાના ઘાતાંકની, અથવા તેનાં પરિણામ ઉમેરાય છે. તેઓના ગુણાંકમાં આવે.

- $682 \times 16 = 10880$

ટૂંકમાં, આ તો આખાયે બીટની, અથવા તેનાં પરિણામ 1010101010 ને 16 (કારણ કે $2^4 = 16$) વડે ગુણવાં જરૂર છે જેમાં 4 એ બેઝિક સંખ્યાના ઘાતાંકની, અથવા તેનાં પરિણામ ઉમેરાય છે. તેઓના ગુણાંકમાં આવે.

Binary angle

How will the representation look if hooked from binary angle
1010101010<<4

Show code

જો આપણે 1010101010 ને આખાયે કોમા સાથે બાઈનરી માં ફેરવવા જઇએ << 4 (ડાબે ઉમેરી) ઓપરેશન કરીએ, તો તેનું પરિણામ નીચેના પ્રમાણે આવશે.

આખાયે દુનિયાનાં સમજૂતી:

૧. મૂળ આખાયે કોમા સાથે: 1010101010 (મૂલ્ય = 682)

૨. શિફ્ટ કરતાં પહેલાં (Left Shift ઓપરેશન): તમામ બીટ્સને ડાબેથી જમણાં તરફ ખસેડવામાં આવે છે અને જમણેથી ખાલી જગ્યાઓ ભરવામાં આવે છે. આમાં 4 નોટેશનલી ઉમેરવાનું છે, તેથી ચાર 0 ઉમેરવામાં આવે છે.

```
Python
મૂળ સ્થાન:           10101010100
4 સ્થાન ડાબે ખસેડ્યું: 10101010100
અંતિમ પરિણામ:        10101010100 (દશાંક: 10880)
```

૩. પરિણામ (અંતિમ): 10880

નોટેશન રીતે કુદરતી ભાષા નિયમ જોઇએ તો 1010101010 ને 16 (કારણ કે $2^4 = 16$) વડે ગુણવાં જરૂર છે જેમાં 4 એ બેઝિક સંખ્યાના ઘાતાંકની, અથવા તેનાં પરિણામ ઉમેરાય છે. તેઓના ગુણાંકમાં આવે.

- $682 \times 16 = 10880$

ટૂંકમાં, આ તો આખાયે બીટની, અથવા તેનાં પરિણામ 1010101010 ને 16 (કારણ કે $2^4 = 16$) વડે ગુણવાં જરૂર છે જેમાં 4 એ બેઝિક સંખ્યાના ઘાતાંકની, અથવા તેનાં પરિણામ ઉમેરાય છે. તેઓના ગુણાંકમાં આવે.

Write a JavaScript function that takes an unsigned integer and returns the number of 1 bits (also known as the Hamming weight). in Gujarati Language

JavaScript માં આ સમસ્યાને ઉકેલવા માટે આપણે તેનાં બાઈનરી સમાન (Binary Equivalent) માં ફેરવવું પડશે અને પછી તેમાં કેટલાં 1 છે તેની ગણતરી કરવી પડશે. આ પ્રક્રિયાને "હેમિંગ વેઈટ" (Hamming Weight) તરીકે ઓળખવામાં આવે છે.

આની રીતે અમારાં ફંક્શન તમે નીચે પ્રમાણે ફેરફારો કરી શકો છો:

JavaScript Code

```
javascript
/*
* Function: countBits(n)
* Purpose: Calculates the number of 1 bits (Hamming weight) for an unsigned integer.
* Gujarati Comment: જો આપેલ સંખ્યા ધન હોય તો તેમાં કેટલાં 1 છે તેની ગણતરી કરે છે.
* Example: countBits(11) -> 3 (કારણ કે 11 નું બાઈનરી રૂપ 1011 છે)
* Author: Gemini
* Date: 2025-10-26
* Version: 1.0
* Dependencies: None
* Usage: const bits = countBits(11);
* Notes: આ ફંક્શન ફક્ત પૂર્ણાંક સંખ્યાઓ સાથે કામ કરે છે.
* Edge Cases: 0 -> 0, 1 -> 1, 2 -> 1, 3 -> 2
*/

function countBits(n) {
  // જો ઇનપુટ સંખ્યા ધન ન હોય તો ભૂલ બતાવો
  if (n < 0) {
    throw new Error("ધન સંખ્યા જરૂરી છે");
  }

  // બાઈનરી સ્ટ્રિંગમાં રૂપાંતર કરો અને '1' અક્ષરોની ગણતરી કરો
  return (n.toString(2).match(/1/g) || []).length;
}

// ઉદાહરણ
console.log(countBits(11) // આઉટપુટ: 3 (કારણ કે 11 નું બાઈનરી રૂપ 1011 છે)
console.log(countBits(0) // આઉટપુટ: 0
console.log(countBits(1) // આઉટપુટ: 1
console.log(countBits(2) // આઉટપુટ: 1
console.log(countBits(3) // આઉટપુટ: 2
```

શું ચાલ્યું તેની સમજૂતી:

- `n.toString(2)`: આ પદ્ધતિ સંખ્યાને તેનાં બેઝ 2 (બાઈનરી) સ્ટ્રિંગમાં રૂપાંતરિત કરે છે. દાખલા તરીકે, અહીં 11 નંબર બાઈનરી સ્ટ્રિંગ '1011' માં રૂપાંતરિત થશે.
- `.match(/1/g)`: આ રેગ્યુલર એક્સપ્રેશન સ્ટ્રિંગમાં '1' અક્ષરોની તમામ ઘટનાઓ શોધે છે. 'g' ફ્લેગ ખાતરી કરે છે કે તે તમામ મેચ શોધે છે, માત્ર પહેલી મેચ નહીં.
- `|| []`: જો `match()` પદ્ધતિ કોઈ મેચ પરત ન કરે (જો સંખ્યામાં કોઈ '1' ન હોય), તો તે ખાલી એરે `[]` પરત કરે છે. આ ભૂલ અટકાવે છે અને ખાતરી કરે છે કે `.length` હંમેશાં કામ કરે છે.
- `.length`: આ પ્રોપર્ટી મેચ કરેલાં તત્ત્વોની સંખ્યા પરત કરે છે, જે આપણે ઇચ્છીએ છીએ તે જ છે.

તમે કેવી રીતે ચકાસી શકો છો કે તે કામ કરે છે:

તમે આ કોડ તમારાં બ્રાઉઝરનાં કન્સોલમાં અથવા Node.js ટર્મિનલમાં ચલાવી શકો છો અને તેનું પરિણામ જોઈ શકો છો.

ઉદાહરણ:

```
$ node -e "console.log(countBits(11))"
3
```

ધ્યાનમાં રાખવાં જરૂરી મુદ્દાઓ:

- **નકારાત્મક સંખ્યાઓ:** આ ફંક્શન નકારાત્મક સંખ્યાઓ માટે ભૂલ ફેંકશે. જો તમે નકારાત્મક સંખ્યાઓ સાથે કામ કરવાં માંગતાં હો, તો તમારે તેનાં પરિણામ અલગ રીતે સંભાળવાં પડશે.
- **બિન-પૂર્ણાંક સંખ્યાઓ:** આ ફંક્શન ફક્ત પૂર્ણાંક સંખ્યાઓ સાથે કામ કરે છે. જો તમારે દશાંક સંખ્યાઓ સાથે કામ કરવાં હોય, તો તમારે તેને પહેલાં પૂર્ણાંકમાં રૂપાંતરિત કરવાં પડશે.
- **કાર્યક્ષમતા:** આ પદ્ધતિ મોટાભાગનાં કિસ્સાઓ માટે ઝડપી અને કાર્યક્ષમ છે.

વૈકલ્પિક અભિગમ (બિટવાઈઝ ઓપરેશન):

જો તમારે આ કરવું જરૂરી નહીં હોય, તો તમે બિટવાઈઝ ઓપરેશનનો ઉપયોગ કરી શકો છો:

JavaScript Code

```
javascript
function countBits(n) {
  if (n < 0) {
    throw new Error("ધન સંખ્યા જરૂરી");
  }
  let count = 0;
  while (n > 0) {
    if (n & 1) {
      count++;
    }
    n >>= 1;
  }
  return count;
}
```

આ વૈકલ્પિક પદ્ધતિ બિટવાઈઝ ઓપરેટર, તે સંખ્યાનાં દરેક બિટ પર પુનરાવર્તન કરે છે અને તેની ગણતરી કરે છે.

ટૂંકમાં, આ ફંક્શન તમારાં પ્રોજેક્ટ માટે કઈ પદ્ધતિ શ્રેષ્ઠ છે તેના પર આધાર રાખે છે.

Given a positive integer, check whether it has alternating bits: namely, if two adjacent bits will always have different values.
Examples 1
Input: n = 5
Output: true

આ પ્રશ્ન ચકાસે છે કે શું આપેલ પૂર્ણાંક તેનાં બાઈનરી રજૂઆતમાં વૈકલ્પિક બિટ ધરાવે છે. આનો અર્થ એ છે કે તમામ 1 નું જૂથ એકલું હોવું જોઈએ. તેઓ એકબીજાની બાજુમાં હોવાં જોઈએ નહીં. આ સમસ્યાને ઉકેલવાની ઘણી રીતો છે. અહીં તેનું એક સરળ અને કાર્યક્ષમ રીત છે.

તર્ક ઉદાહરણ

જો આપણે ઇનપુટ તરીકે 5 નંબર જોઈએ તો આપણે બાઈનરી 101 પર કામ કરીશું. આ પેટર્ન વૈકલ્પિક છે. જો આપણે ઇનપુટ તરીકે 7 નંબર જોઈએ તો આપણે 111 પર કામ કરીશું. આ પેટર્ન વૈકલ્પિક નથી.

વૈકલ્પિક n = 5 (101) અને વૈકલ્પિક n = 101 n & n >> 1 (101 & 10) 101 & 010 101 | 010 = 111 (111 એક સંપૂર્ણ પેટર્ન છે)

જો આપણે આ બે નંબરનાં ઘાતાંકનો ઉમેરો કરીએ તો તેનાં પરિણામમાં બધાં બિટ્સ 1 હશે. આ આપણને જણાવે છે કે મૂળ નંબર વૈકલ્પિક હતો.

Javascript Code

```
javascript
/*
* Function: hasAlternatingBits(n)
* Purpose: Checks if a positive integer has alternating bits.
* Gujarati Comment: શું આપેલ પૂર્ણાંક તેનાં બાઈનરી રજૂઆતમાં વૈકલ્પિક બિટ ધરાવે છે.
* Example: hasAlternatingBits(5) -> true (101)
* Example: hasAlternatingBits(7) -> false (111)
* Example: hasAlternatingBits(10) -> true (1010)
* Author: Gemini
* Date: 2025-10-26
* Version: 1.0
* Dependencies: None
* Usage: const result = hasAlternatingBits(5);
* Notes: આ ફંક્શન ધન પૂર્ણાંક સંખ્યાઓ સાથે કામ કરે છે.
* Edge Cases: 1 -> false, 2 -> true, 3 -> false
*/

function hasAlternatingBits(n) {
  // જો ઇનપુટ ધન ન હોય તો ભૂલ બતાવો
  if (n <= 0) {
    throw new Error("ધન સંખ્યા જરૂરી છે");
  }

  // n & (n >> 1) નો ઉપયોગ કરીને તપાસો કે શું તમામ બિટ્સ વૈકલ્પિક છે
  return (n & (n >> 1)) === 0;
}

// ઉદાહરણ
console.log(hasAlternatingBits(5) // આઉટપુટ: true (101)
console.log(hasAlternatingBits(7) // આઉટપુટ: false (111)
console.log(hasAlternatingBits(10) // આઉટપુટ: true (1010)
```

શીખ અને સમજ:

- `n & (n >> 1)`: આ ઓપરેશન તપાસે છે કે શું તમામ બિટ્સ વૈકલ્પિક છે. જો તમામ બિટ્સ વૈકલ્પિક હશે, તો આ ઓપરેશનનું પરિણામ 0 હશે.
- `n <= 0`: આ ચકાસણી ખાતરી કરે છે કે ઇનપુટ ધન પૂર્ણાંક છે.
- `throw new Error("ધન સંખ્યા જરૂરી")`: જો ઇનપુટ ધન ન હોય તો ભૂલ ફેંકે છે.

શું આપણે આ પદ્ધતિનાં ઉપયોગ કરીને નંબર માટે કોઈ પેટર્ન શોધવાનાં પ્રયાસ કરીશું?

You are given a non-negative integer num and a positive integer k. Your task is to set the k-th bit of the binary representation of num to 1.
The binary representation of num is considered to have its rightmost bit as position 0.
Examples:
Input:
num = 5
k = 1
Output:
7

var setBit = function(n, k) {
};

run this function

પ્રશ્ન સમજો

આપણે બે ઇનપુટ્સ આપવામાં આવ્યાં છે: એક બિન-ઋણાત્મક પૂર્ણાંક (num) અને એક ધન પૂર્ણાંક (k). આપણું કાર્ય k-મા બિટને બાઈનરી રજૂઆતમાં 1 પર સેટ કરવાનું છે. બિટ પોઝિશન 0 થી શરૂ થાય છે, જમણી બાજુથી ગણતરી કરીને.

ઉદાહરણ: num = 5 (101) અને k = 1 પરિણામ: 101 | 010 = 111 (7)

કી આઉટપુટ: 7

તર્ક

આપણે બાઈનરી માસ્ક બનાવવાની જરૂર છે જે ફક્ત k-મા બિટ સિવાય તમામ બિટ્સ પર 0 હોય. આ માસ્ક સાથે ઓરિજિનલ નંબરનું બિટવાઈઝ OR ઓપરેશન કરવું. આ k-મા બિટને 1 પર સેટ કરશે અને અન્ય બિટ્સને અસર કરશે નહીં.

ઉદાહરણ: num = 5 (101) અને k = 1 માસ્ક: 1 << 1 = 2 (010) પરિણામ: 101 | 010 = 111 (7)

કી આઉટપુટ: 7

Javascript Code

```
javascript
/*
* Function: setBit(num, k)
* Purpose: Sets the k-th bit of the binary representation of num to 1.
* Gujarati Comment: આપેલ સંખ્યાનાં બાઈનરી રજૂઆતમાં k-મા બિટને 1 પર સેટ કરે છે.
* Example: setBit(5, 1) -> 7 (101 -> 111)
* Author: Gemini
* Date: 2025-10-26
* Version: 1.0
* Dependencies: None
* Usage: const result = setBit(5, 1);
* Notes: આ ફંક્શન બિન-ઋણાત્મક પૂર્ણાંક સંખ્યાઓ સાથે કામ કરે છે.
* Edge Cases: 0, 1 -> 1, 2 -> 3, 3 -> 7
*/

function setBit(num, k) {
  // જો ઇનપુટ બિન-ઋણાત્મક ન હોય તો ભૂલ બતાવો
  if (num < 0) {
    throw new Error("બિન-ઋણાત્મક સંખ્યા જરૂરી");
  }

  // 1 << k નો ઉપયોગ કરીને માસ્ક બનાવો અને તેનું બિટવાઈઝ OR કરો
  return num | (1 << k);
}

// ઉદાહરણ
console.log(setBit(5, 1) // આઉટપુટ: 7 (101 -> 111)
console.log(setBit(0, 0) // આઉટપુટ: 1
console.log(setBit(1, 1) // આઉટપુટ: 3
console.log(setBit(2, 1) // આઉટપુટ: 7
```

શીખ અને સમજ:

- `num | (1 << k)`: આ ઓપરેશન તપાસે છે કે શું તમામ બિટ્સ વૈકલ્પિક છે. જો તમામ બિટ્સ વૈકલ્પિક હશે, તો આ ઓપરેશનનું પરિણામ 0 હશે.
- `num < 0`: આ ચકાસણી ખાતરી કરે છે કે ઇનપુટ ધન પૂર્ણાંક છે.
- `throw new Error("બિન-ઋણાત્મક સંખ્યા જરૂરી")`: જો ઇનપુટ ધન ન હોય તો ભૂલ ફેંકે છે.

શું આપણે આ પદ્ધતિનાં ઉપયોગ કરીને નંબર માટે કોઈ પેટર્ન શોધવાનાં પ્રયાસ કરીશું?

Write a function that takes an unsigned integer and returns the number of '1' bits (also known as the Hamming weight). in Gujarati Language

આ સમસ્યાને ઉકેલવાની ઘણી રીતો છે. અહીં તેનું એક સરળ અને કાર્યક્ષમ રીત છે. આપણે બાઈનરી રજૂઆતમાં 1 બિટ્સની ગણતરી કરવાની જરૂર છે.

તર્ક ઉદાહરણ

જો આપણે ઇનપુટ તરીકે 11 નંબર જોઈએ તો આપણે બાઈનરી 1011 પર કામ કરીશું. આ પેટર્ન વૈકલ્પિક નથી. આપણે ઇનપુટ તરીકે 7 નંબર જોઈએ તો આપણે 111 પર કામ કરીશું. આ પેટર્ન વૈકલ્પિક નથી.

વૈકલ્પિક n = 5 (101) અને વૈકલ્પિક n = 101 n & n >> 1 (101 & 10) 101 & 010 101 | 010 = 111 (111 એક સંપૂર્ણ પેટર્ન છે)

જો આપણે આ બે નંબરનાં ઘાતાંકનો ઉમેરો કરીએ તો તેનાં પરિણામમાં બધાં બિટ્સ 1 હશે. આ આપણને જણાવે છે કે મૂળ નંબર વૈકલ્પિક હતો.

Javascript Code

```
javascript
/*
* Function: countBits(n)
* Purpose: Calculates the number of 1 bits (Hamming weight) for an unsigned integer.
* Gujarati Comment: આપેલ સંખ્યાનાં બાઈનરી રજૂઆતમાં 1 બિટ્સની ગણતરી કરે છે.
* Example: countBits(11) -> 3 (1011)
* Example: countBits(7) -> 3 (111)
* Example: countBits(0) -> 0
* Author: Gemini
* Date: 2025-10-26
* Version: 1.0
* Dependencies: None
* Usage: const result = countBits(11);
* Notes: આ ફંક્શન બિન-ઋણાત્મક પૂર્ણાંક સંખ્યાઓ સાથે કામ કરે છે.
* Edge Cases: 1 -> 1, 2 -> 1, 3 -> 2
*/

function countBits(n) {
  // જો ઇનપુટ બિન-ઋણાત્મક ન હોય તો ભૂલ બતાવો
  if (n < 0) {
    throw new Error("બિન-ઋણાત્મક સંખ્યા જરૂરી");
  }

  // બાઈનરી સ્ટ્રિંગમાં રૂપાંતર કરો અને '1' અક્ષરોની ગણતરી કરો
  return (n.toString(2).match(/1/g) || []).length;
}

// ઉદાહરણ
console.log(countBits(11) // આઉટપુટ: 3 (1011)
console.log(countBits(7) // આઉટપુટ: 3 (111)
console.log(countBits(0) // આઉટપુટ: 0
console.log(countBits(1) // આઉટપુટ: 1
console.log(countBits(2) // આઉટપુટ: 1
console.log(countBits(3) // આઉટપુટ: 2
```

શીખ અને સમજ:

- `n.toString(2)`: આ પદ્ધતિ સંખ્યાને તેનાં બેઝ 2 (બાઈનરી) સ્ટ્રિંગમાં રૂપાંતરિત કરે છે. દાખલા તરીકે, અહીં 11 નંબર બાઈનરી સ્ટ્રિંગ '1011' માં રૂપાંતરિત થશે.
- `.match(/1/g)`: આ રેગ્યુલર એક્સપ્રેશન સ્ટ્રિંગમાં '1' અક્ષરોની તમામ ઘટનાઓ શોધે છે. 'g' ફ્લેગ ખાતરી કરે છે કે તે તમામ મેચ શોધે છે, માત્ર પહેલી મેચ નહીં.
- `|| []`: જો `match()` પદ્ધતિ કોઈ મેચ પરત ન કરે (જો સંખ્યામાં કોઈ '1' ન હોય), તો તે ખાલી એરે `[]` પરત કરે છે. આ ભૂલ અટકાવે છે અને ખાતરી કરે છે કે `.length` હંમેશાં કામ કરે છે.
- `.length`: આ પ્રોપર્ટી મેચ કરેલાં તત્ત્વોની સંખ્યા પરત કરે છે, જે આપણે ઇચ્છીએ છીએ તે જ છે.

તમે કેવી રીતે ચકાસી શકો છો કે તે કામ કરે છે:

તમે આ કોડ તમારાં બ્રાઉઝરનાં કન્સોલમાં અથવા Node.js ટર્મિનલમાં ચલાવી શકો છો અને તેનું પરિણામ જોઈ શકો છો.

ઉદાહરણ:

```
$ node -e "console.log(countBits(11))"
3
```

ધ્યાનમાં રાખવાં જરૂરી મુદ્દાઓ:

- **નકારાત્મક સંખ્યાઓ:** આ ફંક્શન નકારાત્મક સંખ્યાઓ માટે ભૂલ ફેંકશે. જો તમે નકારાત્મક સંખ્યાઓ સાથે કામ કરવાં માંગતાં હો, તો તમારે તેનાં પરિણામ અલગ રીતે સંભાળવાં પડશે.
- **બિન-પૂર્ણાંક સંખ્યાઓ:** આ ફંક્શન ફક્ત પૂર્ણાંક સંખ્યાઓ સાથે કામ કરે છે. જો તમારે દશાંક સંખ્યાઓ સાથે કામ કરવાં હોય, તો તમારે તેને પહેલાં પૂર્ણાંકમાં રૂપાંતરિત કરવાં પડશે.
- **કાર્યક્ષમતા:** આ પદ્ધતિ મોટાભાગનાં કિસ્સાઓ માટે ઝડપી અને કાર્યક્ષમ છે.

વૈકલ્પિક અભિગમ (બિટવાઈઝ ઓપરેશન):

જો તમારે આ કરવું જરૂરી નહીં હોય, તો તમે બિટવાઈઝ ઓપરેશનનો ઉપયોગ કરી શકો છો:

JavaScript Code

```
javascript
function countBits(n) {
  if (n < 0) {
    throw new Error("બિન-ઋણાત્મક સંખ્યા જરૂરી");
  }
  let count = 0;
  while (n > 0) {
    if (n & 1) {
      count++;
    }
    n >>= 1;
  }
  return count;
}
```

આ વૈકલ્પિક પદ્ધતિ બિટવાઈઝ ઓપરેટર, તે સંખ્યાનાં દરેક બિટ પર પુનરાવર્તન કરે છે અને તેની ગણતરી કરે છે.

ટૂંકમાં, આ ફંક્શન તમારાં પ્રોજેક્ટ માટે કઈ પદ્ધતિ શ્રેષ્ઠ તેના પર આધાર રાખે છે.

var setBit = function(n, k) {
};

fill the logic

પ્રશ્ન સમજો

આપણે બે ઇનપુટ્સ આપવામાં આવ્યાં છે: એક બિન-ઋણાત્મક પૂર્ણાંક (num) અને એક ધન પૂર્ણાંક (k). આપણું કાર્ય k-મા બિટને બાઈનરી રજૂઆતમાં 1 પર સેટ કરવાનું છે. બિટ પોઝિશન 0 થી શરૂ થાય છે, જમણી બાજુથી ગણતરી કરીને.

ઉદાહરણ: num = 5 (101) અને k = 1 પરિણામ: 101 | 010 = 111 (7)

કી આઉટપુટ: 7

તર્ક

આપણે બાઈનરી માસ્ક બનાવવાની જરૂર છે જે ફક્ત k-મા બિટ સિવાય તમામ બિટ્સ પર 0 હોય. આ માસ્ક સાથે ઓરિજિનલ નંબરનું બિટવાઈઝ OR ઓપરેશન કરવું. આ k-મા બિટને 1 પર સેટ કરશે અને અન્ય બિટ્સને અસર કરશે નહીં.

ઉદાહરણ: num = 5 (101) અને k = 1 માસ્ક: 1 << 1 = 2 (010) પરિણામ: 101 | 010 = 111 (7)

કી આઉટપુટ: 7

Javascript Code

```
javascript
/*
* Function: setBit(num, k)
* Purpose: Sets the k-th bit of the binary representation of num to 1.
* Gujarati Comment: આપેલ સંખ્યાનાં બાઈનરી રજૂઆતમાં k-મા બિટને 1 પર સેટ કરે છે.
* Example: setBit(5, 1) -> 7 (101 -> 111)
* Author: Gemini
* Date: 2025-10-26
* Version: 1.0
* Dependencies: None
* Usage: const result = setBit(5, 1);
* Notes: આ ફંક્શન બિન-ઋણાત્મક પૂર્ણાંક સંખ્યાઓ સાથે કામ કરે છે.
* Edge Cases: 0, 1 -> 1, 2 -> 3, 3 -> 7
*/

function setBit(num, k) {
  // જો ઇનપુટ બિન-ઋણાત્મક ન હોય તો ભૂલ બતાવો
  if (num < 0) {
    throw new Error("બિન-ઋણાત્મક સંખ્યા જરૂરી");
  }

  // 1 << k નો ઉપયોગ કરીને માસ્ક બનાવો અને તેનું બિટવાઈઝ OR કરો
  return num | (1 << k);
}

// ઉદાહરણ
console.log(setBit(5, 1) // આઉટપુટ: 7 (101 -> 111)
console.log(setBit(0, 0) // આઉટપુટ: 1
console.log(setBit(1, 1) // આઉટપુટ: 3
console.log(setBit(2, 1) // આઉટપુટ: 7
```

શીખ અને સમજ:

- `num | (1 << k)`: આ ઓપરેશન તપાસે છે કે શું તમામ બિટ્સ વૈકલ્પિક છે. જો તમામ બિટ્સ વૈકલ્પિક હશે, તો આ ઓપરેશનનું પરિણામ 0 હશે.
- `num < 0`: આ ચકાસણી ખાતરી કરે છે કે ઇનપુટ ધન પૂર્ણાંક છે.
- `throw new Error("બિન-ઋણાત્મક સંખ્યા જરૂરી")`: જો ઇનપુટ ધન ન હોય તો ભૂલ ફેંકે છે.

શું આપણે આ પદ્ધતિનાં ઉપયોગ કરીને નંબર માટે કોઈ પેટર્ન શોધવાનાં પ્રયાસ કરીશું?

વૈકલ્પિક, આ સોલ્યુશન વૈકલ્પિક બિટ્સની ગણતરી **Brian Kernighan's Algorithm** (બ્રાયન કર્નિઘન્સ અલ્ગોરિધમ) નો ઉપયોગ કરીને કરે છે. આ અલ્ગોરિધમ ખૂબ જ કાર્યક્ષમ છે કારણ કે તે દરેક બિટ પર પુનરાવર્તન કરવાને બદલે ફક્ત સેટ બિટ્સ પર પુનરાવર્તન કરે છે.

ટૂંકમાં, આ ફંક્શન તમારાં પ્રોજેક્ટ માટે કઈ પદ્ધતિ શ્રેષ્ઠ તેના પર આધાર રાખે છે.

ઉદાહરણ

જો આપણે ઇનપુટ તરીકે 11 નંબર જોઈએ તો આપણે બાઈનરી 1011 પર કામ કરીશું. આ પેટર્ન વૈકલ્પિક નથી. આપણે ઇનપુટ તરીકે 7 નંબર જોઈએ તો આપણે 111 પર કામ કરીશું. આ પેટર્ન વૈકલ્પિક નથી.

વૈકલ્પિક n = 5 (101) અને વૈકલ્પિક n = 101 n & n >> 1 (101 & 10) 101 & 010 101 | 010 = 111 (111 એક સંપૂર્ણ પેટર્ન છે)

જો આપણે આ બે નંબરનાં ઘાતાંકનો ઉમેરો કરીએ તો તેનાં પરિણામમાં બધાં બિટ્સ 1 હશે. આ આપણને જણાવે છે કે મૂળ નંબર વૈકલ્પિક હતો.

Javascript Code

```
javascript
/*
* Function: countBits(n)
* Purpose: Calculates the number of 1 bits (Hamming weight) for an unsigned integer.
* Gujarati Comment: આપેલ સંખ્યાનાં બાઈનરી રજૂઆતમાં 1 બિટ્સની ગણતરી કરે છે.
* Example: countBits(11) -> 3 (1011)
* Example: countBits(7) -> 3 (111)
* Example: countBits(0) -> 0
* Author: Gemini
* Date: 2025-10-26
* Version: 1.0
* Dependencies: None
*
```

એકવાર, આ સંખ્યાને દ્વિઅંક માટે આગળની **Bitwise OR** (`|`) અને **Left Shift** (`<<`) ઓપરેટર દ્વારા પ્રક્રિયા કરીએ.

JavaScript Code

```
javascript

// Bitwise Number 1 - શૂન્ય ઉમેરો
// Bitwise Number 2 - ત્રણ ડાબેથી વિસ્થાપન કર્યા પછી શૂન્ય ઉમેરો
// Bitwise Number3

var setBit = function(n, k) {
  // કોષ્ટક 1 માટે શૂન્ય સાથે બાઈનરી ઓર કરવું
  // કોષ્ટક 2 માટે ડાબેથી ત્રણ સ્થાનો ખસેડ્યા પછી શૂન્ય સાથે બાઈનરી ઓર કરવું
  let mask = 1 << k;
  return n | mask;
};

// ઉદાહરણ:
console.log(setBit(5, 1));

// આઉટપુટ: 7
```

આ તકનીકમાં શું થઈ રહ્યું છે? (પગલું-દ્વારા)

ચાલો `5 | (1 << 1)` જુઓ.

૧. `1 << 1` ખસેડવાનું કામ કરશે. `1` ના `1` બિટ દ્વારા ડાબેથી ૧ સ્થાન ખસેડવાથી પરિણામ આવશે `10` (દશાંક ૨).

```
Pattern
  001  (1)
| 010  (1<<1)
  ---
  011  (દશાંક 3)
```

મૂળ સંખ્યાની જેમ જ બંને બાઈનરી જુઓ. હવે, દરેક બિટ સ્થાન પર, તપાસો કે શું ઓછામાં ઓછું એક `1` છે (લીલો). આ ઘટના `1` ધરાવતી દરેક સ્થિતિ માટે સત્ય છે. પરિણામ `011` છે, જે દશાંક `3` છે.

શું તમે જાણતા હતા કે તમે ઓપરેટર તરીકે ડિફોલ્ટ કિંમત કેવી રીતે આપી?

You are given a non-negative integer num and a position k. Your task is to set the bit at position k in the binary representation of num to 1.

The binary representation of num is considered to have its lightmost bit as position 0.

Input Format:

An integer num representing the given number.

An integer k representing the position of the bit to be set.

Output Format:

An integer representing the result after setting the k-th bit.

Example:

Input:
num = 5
k = 1
Output:
7

var setBit = function(n, k) {

}

run the Javascript logic in this function

એકવાર, આ તકનીકમાં ખસેડવાના ઉદાહરણમાં, મેં શૂન્ય ઉમેરવાનું કામ કર્યું. શું તમારા ગણિત શિક્ષકે તમને કહ્યું હતું કે તમે ડાબેથી ત્રણ સ્થાનો ખસેડ્યા પછી શૂન્ય ઉમેરવાનું કામ કર્યું? ના, તમારા ગણિત શિક્ષકે તમને કહ્યું હતું કે તમે જમણેથી ત્રણ સ્થાનો ખસેડ્યા પછી શૂન્ય ઉમેરવાનું કામ કર્યું. આ

Step 1	Nested Linear Loops	$O(N^2)$
Step 2	Linear + Logarithmic	$O(N \log N)$

Export to Sheets

તું તમારે આ ટેબ્લેટ પર Space Complexity શું આવશે? સમજાવો છો?

Which of the given options provides the increasing order of complexity of functions f_1, f_2, f_3 and f_4 for large values of n and why ?

$f_1(n) = 2^n n$

$f_2(n) = n^{100}(2)$

$f_3(n) = n^{100}(3)$

$f_4(n) = n \log n$

Choose the correct Option

a) f_1, f_2, f_3, f_4
b) f_2, f_3, f_1, f_4
c) f_4, f_2, f_3, f_1
d) f_4, f_3, f_2, f_1

Not Attempted

હમણાં ચિત્રમાં f_1, f_2, f_3, f_4 જોઈ.

પહેલો પ્રશ્ન પૂછે છે કે કયો ઝડપથી ખરાબ (Complexity) વધારે થાય અને વધારામાં વધી જાય? તો ઉદાહરણ તરીકે (Growth rate) કયો ઝડપથી વધે.

સૌથી ખરાબ કોણ લાગુ પડે?

$f_1(n) = n \log n$ ઝડપથી મોટી પછી $f_2(n)$ આ તો એક Linearithmic (લગભગ તો n^2 કરતાં સારું) પછી પણ લીનિયર n^{100} આવે છે. n કરતાં લાગુ થઈ ગયો છે. તો આ ચિત્રમાં f_4 સૌથી નીચું આવ્યા છે.

$f_2(n) = n^{100}$ અને n^{100} આ તો Polynomial (કાત) છે. તો આ n કરતાં જ n લાગુ થયા. ખરાબી તો તો ઝડપથી n કરતાં n^2 આવ્યા એટલે તો f_2 આવ્યા એટલે તો તો.

$f_3(n) = n^{100}$ અને n^{100} આ તો Polynomial (કાત) છે. તો આ n કરતાં જ n લાગુ થયા. ખરાબી તો તો ઝડપથી n કરતાં n^2 આવ્યા એટલે તો f_3 આવ્યા.

$f_1(n) = 2^n n$ સૌથી નહું ખરાબ છે. તો એક Exponential (કાત) છે. તો એક n કરતાં ઘણાં વધારે ઝડપથી ખરાબ થઈ ગયા (Polynomial બધાને). Quasi polynomial આવ્યા અહીં આવી ગયા છે.

ચડિયાતો ક્રમાવો જો (Increasing Order):

1. f_4 $n \log n$
2. f_2 n^{100}
3. f_3 n^{100}
4. f_1 2^n

હવે, ત્યારે જો $f_1 < f_2 < f_3 < f_4$ છે.

તું શું કહી શકે આ ટ્રાન્સફર થતાં $n = 100$ જેવું કિંમત મૂકીને સમજાવી શકીએ? આવશે?

What Will be the Time Complexity of the following Questions:

```
for (int i = 1; i <= n; i*= 2) {
    // Some operation
}
```

તો લોકો Time Complexity (TC)નું શું થશે.

આવું કરે તોય ખરાબ લોગ આવે ખરાબમાં જ છે.

3. મૂળભૂત વિશ્લેષણ (Analysis)

આ મૂળભૂત વિશ્લેષણ $\Rightarrow$ કે લીનિયર કરીને લોગમાં આવી ટ્રાન્સફર થઈ ગયો.

- લૂપ્સનું વિશ્લેષણ $\Rightarrow$
- નેસ્ટેડ લૂપ્સ $i = 1 \rightarrow 5 \rightarrow 25$
- નેસ્ટેડ લૂપ્સ $i = 2 \rightarrow 2 \rightarrow 4$
- લૂપ્સ સ્ટેપ્સ $i = 1 \rightarrow 2 \rightarrow 8$
- આ લૂપ્સ તો મૂળભૂત ઓપરેશન લાગુ પડે i નહીં.

4. ગણિતીય રીતે સમજાવું. (Mathematical Proof)

પછી તે આ મૂળભૂત તે ગણિત થઈને k સુધીનો પછી 2^k થઈ.

તું શું કહી શકાવે ખરાં? $2^k = n$ થયા. તોજ, આપણે લોગમાં જઈ શકીએ છે.

$$2^k = n$$

હવે અંતે આવું Logarithm (લોગ) થઈ.

$$\log_2(2^k) = \log_2(n)$$

$$k \cdot \log_2(2) = \log_2(n)$$

$$k = \log_2(n)$$

(અહીં $k \log_2(2) = 1$)

5. નિષ્કર્ષ

ખરાં પણ જોઈ શું થયું? કોઈપણ મૂળભૂત ઓપરેશન (multiplication) ગુણાકાર કરીએ તોય ખરાબ ન્યાયસર (Division) હોય. પછી કોઈ પણ લીનિયર કરીએ. Logarithmic કરી જ.

હોવું, આ હોવું Time Complexity (TC)નું શું થશે.

તું શું જણાવ્યા વગર જો કોઈ $\Rightarrow$ ખરાબ $\frac{1}{2}$ ખરાબ $\frac{1}{2}$ કરતાં $\frac{1}{2}$ કરતાં $\frac{1}{2}$ કરતાં $\frac{1}{2}$ કરતાં $\frac{1}{2}$ કરતાં

What Will be the Time Complexity of the following Questions:

```
for (int i = 1; i <= n; i*= 2)
    for (int j = 1; j <= n; j*= 2)
        for (int k = 1; k <= n; k*= 2)
            // Some operation
}
}
}
```

આ લોગનો Time Complexity ($O(n^3)$) (Cubic Complexity) થશે.

આવું કરે તોય ખરાં પણ સૌથી ખરાબમાં જ છે.

7. મૂળભૂત વિશ્લેષણ (Analysis)

આવી રીત કોઈ પણ ખરાબમાં જુએ છે, જે નોસ્ટેડમાં ખરાં છે.

- નેસ્ટેડ લૂપ્સનું વિશ્લેષણ (Nested Loop - 1) આ જુએ છે $n \times n$ - 1 જુઓ અંતે તો કરતાં $\frac{1}{2}$ કરતાં $\frac{1}{2}$ કરતાં $\frac{1}{2}$ કરતાં
- વેરીએબલ લૂપ્સ (Variable Loop - 2) આ જુએ છે $i \times n$ - 1 જુઓ અંતે તો n કરતાં જુઓ તોય તોય i કરતાં જુઓ તોય
- નેસ્ટેડ વેરીએબલ લૂપ્સ (Inner Loop - 3) આ જુએ છે $i \times n$ - 1 જુઓ અંતે તો n કરતાં જુઓ તોય i કરતાં જુઓ તોય i કરતાં જુઓ તો છે.

8. કુલ ઓપરેશનો ગણવાં (Total Operations)

ખરાં જુઓ ગણો તોય, તમે તેનાં કુલ ઓપરેશનો ગુણાકાર થાય છે.

$$\text{Total Iterations} \Rightarrow (\text{outer loop}) \times (\text{middle loop}) \times (\text{inner loop})$$

$$\text{Total Iterations} \Rightarrow n \times n \times n = n^3$$

9. નિષ્કર્ષ

ઓકમાં તે ખરાબમાં સર્વોચ્ચ (Worst operation) ગુણાકાર n^3 ખરાં થાય છે. તોજ Big-O notation ગુણાકાર કરીશું Time Complexity ($O(n^3)$) થઈશાય.

નોલોગન હોવું, એ ખરાબ ઘણાં તે ખરાબમાં ખરાબ ગુણાકાર કરતાં ગુણાકાર કરતાં, તો તરત લગભગલાગ $O(n^3)$ થઈ જશે.

તું શું જાણ જો કોઈએ જો ઘણાંને ખરાં કરતાં ખરાં કરતાં ખરાબમાં કરતાં (Nested loop) કરતાં, તો તું ફોરાન ખરાં તે ખરાબ પહેલે જોઈ?

What Will be the Time Complexity of the following Questions:

```
for (int i = 1; i <= n; i*= 2)
    for (int j = 1; j <= n; j*= 2)
        for (int k = 1; k <= n; k*= 2)
            // Some operation
}
}
}
```

આ લોગનો Time Complexity ($O(n^3)$) (Cubic Complexity) થશે.

આવું કરે તોય ખરાં પણ સૌથી ખરાબમાં જ છે.

7. મૂળભૂત વિશ્લેષણ (Analysis)

આવી રીત કોઈ પણ ખરાબમાં જુએ છે, જે નોસ્ટેડમાં ખરાં ત

સૌથી ઓછું ગુણાકાર ધરાવતા 2 થી ∞ સુધી તમામ પૂર્ણાંકો શોધવા માટે તમારે એક સંખ્યા તરીકે 1 થી શરૂ કરીને તમામ પૂર્ણાંકો તપાસવા પડશે. જ્યારે તમે એક સંખ્યા શોધો કે જેનો ગુણાકાર 2 હોય અને તે 2, 3 કે તેનાથી મોટી કોઈપણ સંખ્યાથી વિભાજિત ન થાય, ત્યારે તે સંખ્યા પછી તમામ તપાસ કરવાની જરૂર નથી કારણ કે તે તમામ પૂર્ણાંકોનો ગુણાકાર હશે અને તેથી તે 2 ની તમામ ગણતરી કરશે.

ઉદાહરણ

મહત્તમ સંખ્યાની ગણતરી 1 થી ∞ સુધી તે સંખ્યા સંખ્યા 2 થી ∞ : 1 છે. આપણે સંખ્યા તરીકે 1 નો ઉપયોગ કરીએ છીએ કારણ કે તે ગુણાકાર, અવયવ કરતા વધારાના ગુણાકાર ધરાવતા નથી. જો તે તમામ પૂર્ણાંકો તપાસવામાં આવે તો તે 2 ની તમામ ગણતરી કરશે.

Javascript Code

```javascript
/
* Execute Function()
* Execute Function()
/

var FindMissingNumbers = function(num) {
    let n = num.length;
    let result = [];

    // સંખ્યા શૂન્ય તરીકે શરૂ કરો (ઓફ સેટ)
    for (let i = 0; i < n; i++) {
        // સંખ્યાને 1 તરીકે સેટ કરો (ગુણાકાર ઓપરેટર માટે તટસ્થ કિંમત 1 પર પ્રારંભ કરીને સંખ્યા શોધી શકે છે)
        let index = Math.abs(num[i]) - 1;

        // જો આ ઇન્ડેક્સ પહેલાં જોવામાં આવ્યો નથી, તો તેને નકારીને કરો
        if (num[index] > 0) {
            num[index] = -num[index];
        }
    }

    // બીજા પાસ: સકારાત્મક ઇન્ડેક્સ ધરાવતા તમામ ઇન્ડેક્સ ઉમેરો (મૂળ મૂલ્યો તપાસો)
    for (let i = 0; i < n; i++) {
        if (num[i] > 0) {
            result.push(i + 1);
        }
    }

    return result;
};

// ઉદાહરણ
let num = [4, 3, 2, 7, 8, 2, 3, 1];
console.log(FindMissingNumbers(num));// આઉટપુટ: [5, 6]
```

આ કોડ કેટલા સમય લેશે? કઠોરતા અને સ્પેસ જટિલતા સમજાવો

આપણે તેને `num = [4,3,2,7,8,2,3,1]` તરીકે ધ્યાનમાં રાખીશું.

1. મોટેભાગે કઠોરતા:

- પ્રથમ લૂપ માટે 8 – 0 = 8 ઇન્ડેક્સ 2 પર્યંત તપાસવા ઓપરેશન અને `-num[i]` જ્યાં 2 અને 3 – 7 અને 1 – 7 અને
- બીજો લૂપ 8 થી – 0 = 8 ઇન્ડેક્સ 2 પર્યંત તપાસવા ઓપરેશન અને `num[i]` જ્યાં 2 અને 3 – 7 અને
- આ રીતે મોટેભાગે ચલાવવામાં આવતા જટિલતા સંબંધિત 8 ઓપરેશન છે. ઓપરેશન ઉમેરવાની સંખ્યાની.

2. જેમાં સંખ્યાનું પુનરાવર્તન કરવામાં આવે છે તેવા કિસ્સામાં કઠોરતા: 2 અને 3 પુનરાવર્તિત છે તેથી 2 થી 3 સુધી

- ઇન્ડેક્સ 2 પર 8 ઓપરેશન છે – ઇન્ડેક્સ 3 પર્યંત 8.
- ઇન્ડેક્સ 3 પર 8 ઓપરેશન છે – ઇન્ડેક્સ 4 પર્યંત 8.

Complexity Analysis:

- **Time Complexity:** O(n). આપણે સંખ્યાને બે વાર શૂન્ય ઇન્ડેક્સ સુધીમાં તપાસીએ છીએ.
- **Space Complexity:** O(1). આપણે મૂળ સંખ્યાને જ ઉપયોગ કરી રહ્યા છીએ. (ઇનપુટ સંખ્યાને બાદબાકીમાં ઉમેરવા સિવાયનું નથી).

શું તમારી પાસે જે સંખ્યાઓનું ઉમેરવાના કોઈપણ ડુપ્લicેટ હોય તો કોડ સંશોધિત કરો જવાબનું ફરી?

```
var missing = function(nums, n) {
    let n = nums.length;
    let result = [];
    // સંખ્યાને 1 તરીકે સેટ કરો (ઓફ સેટ)
    for (let i = 0; i < n; i++) {
        let index = Math.abs(nums[i]) - 1;
        if (nums[index] > 0) {
            nums[index] = -nums[index];
        }
    }

    for (let i = 0; i < n; i++) {
        if (nums[i] > 0) {
            result.push(i + 1);
        }
    }

    return result;
}

Error:
The error message Referenced1nor missing elements is not defined indicates that the function missing elements is being called, but a name is been defined in your code. The problem description asks for the missing elements in the array, but your code defines a name function. You should rename your function to missing_elements and ensure it is correctly implemented to find the missing numbers.
```

તમારા કોડમાં જે ભૂલ મળી છે તેનું કારણ એ છે કે તમારું નામ ફંક્શન `missing` અસ્તિત્વમાં છે. તેને ફંક્શન મૂલ્યોની કોઈ નામ સોંપવાની જરૂર છે.

સુધારેલો Javascript કોડ

```javascript
/
* Execute Function()
* Execute Function()
/

var missing_elements = function(nums) {
    let n = nums.length;
    let result = [];

    // સંખ્યાને 1 તરીકે સેટ કરો (ઓફ સેટ)
    for (let i = 0; i < n; i++) {
        let index = Math.abs(nums[i]) - 1;
        if (nums[index] > 0) {
            nums[index] = -nums[index];
        }
    }

    for (let i = 0; i < n; i++) {
        if (nums[i] > 0) {
            result.push(i + 1);
        }
    }

    return result;
};

// ઉદાહરણ
let num = [4, 3, 2, 7, 8, 2, 3, 1];
console.log(missing_elements(num));// આઉટપુટ: [5, 6]
```

તમારા કોડમાં કઠોરતાની ગણતરી કરવામાં આવી છે:

1. ઇનપુટ: અમને એક પૂર્ણાંક મૂલ્ય, તમારી ઇનપુટ `missing_elements` તરીકે ઇનપુટ કરીને છે. પણ કાર્ય કોલ `missing` નામ આપવું જોઈએ. જે કોડ આપેલો હતો તે મુજબ.

2. ઓપરેટર્સ (Nested Loop): તમારા કોડમાં તમે બીજા લૂપમાં પહોંચી જાવ ત્યાં સુધી તેની તપાસ કરવામાં આવતા કોડ સંખ્યા પર આવતી નથી કારણ કે તમારી પૂર્ણ સંખ્યા શૂન્ય કરતાં મોટી છે. પરંતુ તે પૂર્ણ સંખ્યા શૂન્ય કરતાં મોટી નથી તેવા કિસ્સામાં, જ્યારે તે પૂર્ણ સંખ્યા શૂન્ય કરતાં મોટી નથી તેવા કિસ્સામાં, તે બીજા લૂપ પર જશે.

3. સંભવિત જટિલતા (Complexity): તે તમારી ઇનપુટ `num[i]` અથવા તેનું મૂલ્ય શૂન્ય કરતાં મોટું છે કે નહીં તેની તપાસ કરવામાં આવતી સંખ્યાની સંખ્યા `num[i]` જેટલા સમયની ઝડપે છે.

આ કોડ કેટલો સમય લેશે?

- સૌથી ખરાબ સંજોગોમાં કોડને તમામ ઇનપુટ તપાસવાની જરૂર પડશે. જો આપણને 2 હતો, તો આપણને તમામ 2 પર જવાની તપાસ કરવાની જરૂરિયાત પડશે તેવી સંભાવના. આ "સમયની ગુણવત્તા" તરીકે ઓળખાય છે.
- સૌથી ઝડપી આપણે જોઈ શકાતાં જે આપણે કઠોરતાની તપાસ કરવાની જરૂર પડશે તેવી સંખ્યા. તે કોડ પર્યંત તે કોડ તમામ સંખ્યાને તપાસ આપશે નહીં.

શું તમારી પાસે જે ઓપરેટર પર? Space Complexity જણાવો **Time Complexity** વિશે પણ જણાવવું જોઈ?

Operator can make mistakes, so double-check it

Operator can make mistakes, so double-check it